Basic PINN – Architecture

Introduction

Physics Informed Neural Network (PINN) is a Machine Learning (ML) model to train an Artificially Intelligent System to solve Physics Equations on its own by relying on the information given to it while training it. For example, consider a simple case in following equation

$$\frac{\partial^2 \psi}{\partial x^2} = \frac{1}{v^2}\frac{\partial^2 \psi}{\partial t}$$

which is a 1-Dimensional second order Partial Differential Equation (PDE) representing for example oscillations on a string, where each point on the string oscillates, about the mean, point with velocity v and the two ends ($x = 0$ and $x = L$) of the strings are fixed as per the following boundary conditions

$$\psi(0, t) = 0 \text{ and } \psi(L, t) = 0$$

The solution to this equation is given as follows:

$$\psi(x, t) = \left[A_n \cos\left(\frac{n\pi v t}{L}\right) + B_n \sin\left(\frac{n\pi v t}{L}\right)\right] \cdot \sin\left(\frac{n\pi x}{L}\right)$$

or

$$\psi(x, t) = \psi(t) \cdot \psi(x)$$

- $\psi(x, t)$ is the amplitude (vertical displacement) of each point on the string
- $A_n$and B_n are coefficients depending on the amplitude of the oscillations and n is the mode of oscillations or harmonics.

Now, the method we use generally to solve this PDE is separation of variable. But out aim is to solve this equation (or equations of similar kind with some perturbations) using an ML model, i.e. PINN, and obtain the same solution. Once we have a trained model which can

solve this equation, we can use it to solve equations of similar kind and automate the task of equation solving.

Making of PINN-ML Model (4 Pillars Approach)

Following is the description of the architecture for a general PINN ML Model.

Step 1: Import important libraries

```
import torch
import torch.nn as nn
import torch.optim as optim
import torch.autograd as autograd
import numpy as np
import time
import matplotlib.pyplot as plt
```

i. Torch library is used for making the neural network
ii. Some functions/methods from the torch library are imported explicitly and they will be described when they're used
iii. Numpy library is used for handling the mathematics as well as data management and analysis part.
iv. Time library is used for noting the time taken to train the model
v. Matplotlib.pyplot is imported (as variable plt) to plot graphs (if required)

Step 2: Predefining parameters

We use float datatype for all variables as we are solving a mathematical equation. Secondly, since random points are used to train the model, we need to set a seed to choose random points according to a seed whenever we train.

```
torch.set_default_dtype(torch.float)
torch.manual_seed(1234)
np.random.seed(1234)
```

Step 3: Defining data points (Pillar 1)

In this step we define the variables and parameters that are involved in the mathematical equation. In this case we need to define the data for x, t, u where u is my function that depends on x and t.

i. First we define x and t and make a mesh for x and t. We can also define the lower and upper bounds on u.

```
x = np.linspace( start: 0,  stop: 10,  num: 201)
t = np.linspace( start: 0,  stop: 100,  num: 201)
X, T = np.meshgrid(x, t)
```

The structure of the mesh for X and T is

```
[[ 0.    0.05  0.1  ...  9.9   9.95 10.  ] x values for t = 0
 [ 0.    0.05  0.1  ...  9.9   9.95 10.  ] x values for t = dt
 [ 0.    0.05  0.1  ...  9.9   9.95 10.  ] x values for t = 2dt
 ...                                        and so on
 [ 0.    0.05  0.1  ...  9.9   9.95 10.  ]
 [ 0.    0.05  0.1  ...  9.9   9.95 10.  ]
 [ 0.    0.05  0.1  ...  9.9   9.95 10.  ]] x values for t = T
```

```
t values are along the column for each x value
First column is t values for x = 0 and so on
[[  0.    0.    0.  ...   0.    0.    0. ]
 [  0.5   0.5   0.5 ...   0.5   0.5   0.5]
 [  1.    1.    1.  ...   1.    1.    1. ]
 ...
 [ 99.   99.   99.  ...  99.   99.   99. ]
 [ 99.5  99.5  99.5 ...  99.5  99.5  99.5]
 [100.  100.  100.  ... 100.  100.  100. ]]
```

ii. Then, inside a function (say training data), we make arrays for different kinds for values for x, t and u, namely the initial value and boundary value arrays for u and their corresponding arrays for x and t. In the bounded string example, the two ends are fixed. So in this case arrays to be defined are as follows

```python
def training_data():
    left_boundary = np.hstack((X[:, 0][:, None], T[:, 0][:, None]))
    right_boundary = np.hstack((X[:, -1][:, None], T[:, 0][:, None]))
    u_left = np.zeros(shape=(left_boundary.shape[0], 1))
    u_right = np.zeros(shape=(right_boundary.shape[0], 1))
```

Also include the initial condition and syntax:

```python
initial_condition_x = np.hstack((X[0, :][:, None], T[0, :][:, None]))
u_initial = np.sin(np.pi * X[0, :][:, None])
```

Their structure are like the following

```
For x = 0 and T = [0 to T]
[[  0.    0. ] (0, 0)
 [  0.    0.5] (0, dt)
 [  0.    1. ] (0, 2dt)
       .
       .      and so on
       .
 [  0.   99. ]
 [  0.   99.5]
 [  0.  100. ]] (0, T)
```

```
For x = L and T = [0 to T]
[[ 10.    0. ] (L, 0)
 [ 10.    0.5] (L, dt)
 [ 10.    1. ] (L, 2dt)
       .
       .      and so on
       .
 [ 10.   99. ]
 [ 10.   99.5]
 [ 10.  100. ]] (L, T)
```

```
For right boundary
[[0.] u(L, 0)
 [0.] u(L, dt)
 [0.] u(L, 2dt)
  .
  .    and so on
  .
 [0.]
 [0.]
 [0.]] u(L, T)
```

These are the boundary points where we find the boundary loss.

iii. Next, we randomly choose some points from X and T mesh for the collocation points. At these points we calculate the physics loss.

```python
x_flat = X.flatten()[:, None] # Shape: (40401, 1)
t_flat = T.flatten()[:, None] # Shape: (40401, 1)

# 2. Stack them to create the full domain of points
X_test = np.hstack((x_flat, t_flat)) # Shape: (40401, 2)

N_f = 10000 # Number of collocation points

# Generate N_f unique random indices to select from an X_test
idx = np.random.choice(X.shape[0], N_f, replace=False)

# Select the actual coordinate pairs
X_f_train_raw = X_test[idx, :]
```

The value of the function u at these points will be calculated by the neural network.

iv. Finally we convert all the arrays into tensor data type (valid for torch library) and return. The syntax for that is: torch.from_numpy.(array).float()

Step 4: The Neural Network (Pillar 2)

This is the "Brain" that tries to guess the solution $u(x, t)$. Here we define the structure of the model and training process

```
class PINN_Model(nn.Module):
    def __init__(self):
        super().__init__()
        # STEP 1: Define the layers (Width and Depth)
        self.fc1 = nn.Linear( in_features: 2,  out_features: 50)  # Input is 2 (x, t)
        self.fc2 = nn.Linear( in_features: 50,  out_features: 50)
        self.fc3 = nn.Linear( in_features: 50,  out_features: 50)
        self.fc4 = nn.Linear( in_features: 50,  out_features: 1)  # Output is 1 (u)

        # STEP 2: Choose a smooth activation function
        self.activation = nn.Tanh() # or relu, or sinh and so on

        # STEP 3: Define loss function
        self.loss_function = nn.MSELoss(reduction='mean')

    def forward(self, x_t):  1 usage
        # STEP 1: Normalize both x and t before training
        ....
        # STEP 2: The Propagation
        a = self.activation(self.fc1(x))
        a = self.activation(self.fc2(a))
        a = self.activation(self.fc3(a))
        a = self.fc4(a)  # No activation on the final layer
        return a
```

Inside the innit method we define the variables of the class and the forward method is the one where model is trained (in this case using 4 layers)

Step 5: The PINN Engine (Pillar 3)

In this step, we calculate the loss or error at the boundary points (boundary loss) and at the collocation points (physics loss) and do the weighted sum of these two losses.

```
def loss(self, collocation_points, boundary_points, u_boundary_values):
    # Make a clone for collocation points
    cp = collocation_points.clone()
    cp.requires_grad = True

    # calculate u using forward method at collocation and boundary points
    u_physics = self.forward(cp)
    u_boundary = self.forward(boundary_points)

    # calculate derivatives (if any) for the equation using autograd
    u_x_t = autograd.grad(u_physics.sum(), cp, retain_graph=True, create_graph=True)[0]
    u_x = u_x_t[:, [0]]  # Derivative w.r.t x
    u_t = u_x_t[:, [1]]  # Derivative w.r.t t

    u_xx_x_t = autograd.grad(u_x.sum(), cp, create_graph=True)[0]
    u_xx = u_xx_x_t[:, [0]] # Second derivative w.r.t. x

    u_tt_x_t = autograd.grad(u_t.sum(), cp, create_graph=True)[0]
    u_tt = u_tt_x_t[:, [0]]  # Second derivative w.r.t. t

    # Define the equation and calculate physics loss and boundary loss
    f = u_xx - (u_tt / (v ** 2))

    # Physics Loss (Goal: f should be zero)
    f_hat = torch.zeros(f.shape[0], 1)
    loss_physics = self.loss_function(f, f_hat)

    # Boundary loss
    loss_boundary = self.loss_function(u_boundary, u_boundary_values)
    return 10 * loss_physics + .1 * loss_boundary
```

Step 6: Training (Pillar 4)

Usually, it is better to reserve some points for training the model and the rest for testing the model. To train the model, just keep on calculating

the loss value for new set of weights and biases and model parameters until the model is trained.

```
start_time = time.time()

max_iter = 3000
for i in range(max_iter):
    loss = PINN.loss(X_u_train, u_train, X_f_train)
    optimizer.zero_grad()  # zeroes the gradient buffers of all parameters
    loss.backward()  # backprop
    optimizer.step()
    if i % 200 == 0:
        print(loss)

elapsed = time.time() - start_time
print('Training time: %.2f' % (elapsed))
```

Finally the model is trained. Now to find the value of u at any point, just call the forward function of the PINN_Model class and that would be the predicted value of u at an point.

**optimizer = optim.Adam(pinn.parameters(), lr=1e-2)